



National University
of computer and emerging sciences

CS3002 - Design and Analysis of Algorithm

Project Report

Instructor

Sir Waqas Sheikh

Group Member

Noman Vadsariya (19K-1432)

M.Subhan Khan (19k-1285)

Abstract

This project implements seven different graph algorithms and tests these algorithms on given benchmark of files with increasing number of nodes (from 10 to 100 nodes). These graph algorithms include three minimum spanning tree algorithms (Prim's, Kruskal and Boruvka's algorithm) and three shortest path algorithms (Dijkstra, Bellman Ford and Floyd-Warshall).

Introduction

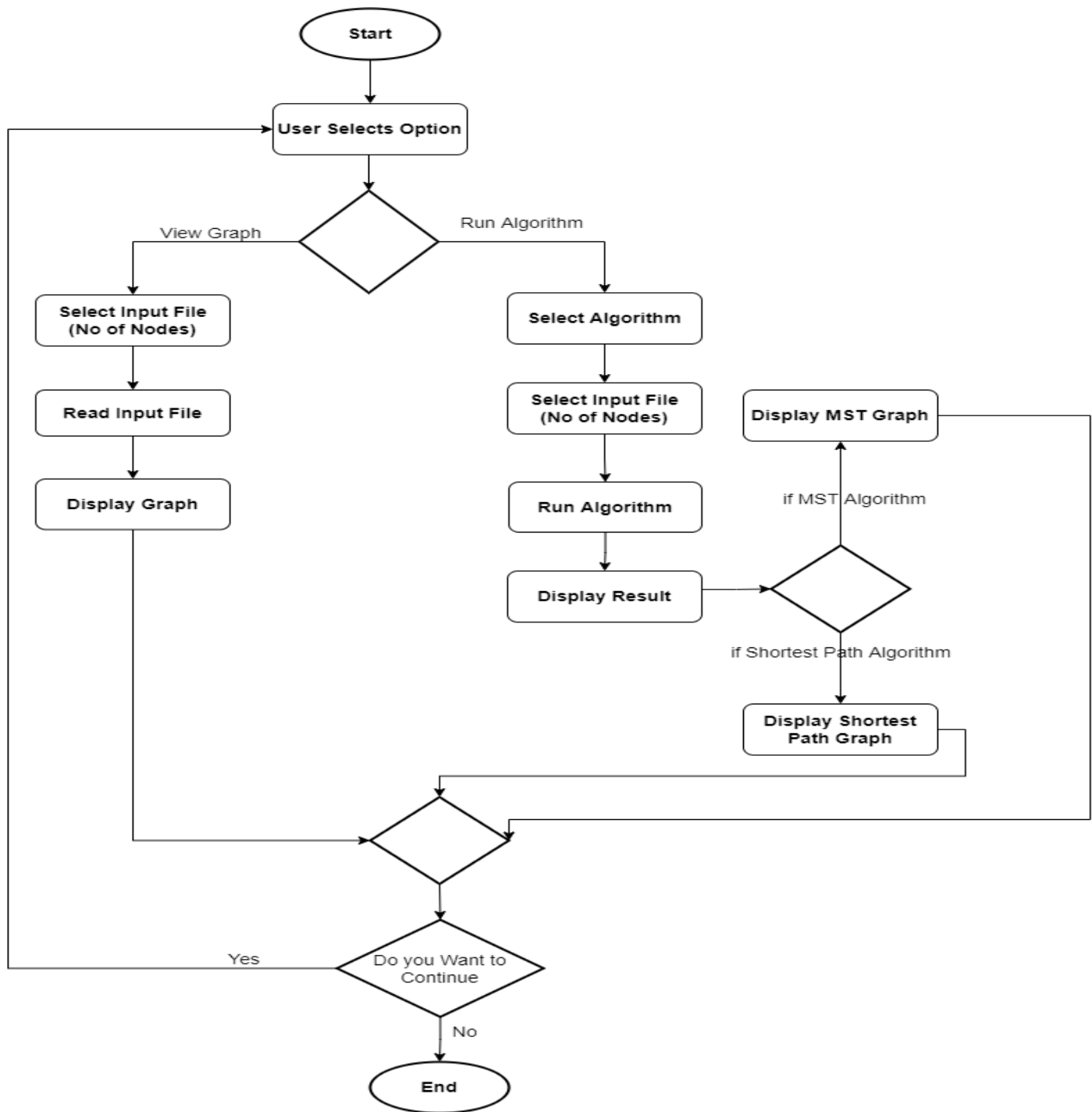
Graphs have become a powerful means of modelling and capturing data in real-world scenarios such as social media networks, web pages and links, and locations and routes in GPS. If you have a set of objects that are related to each other, then you can represent them using a graph. Graphs can be implemented in different ways such as adjacency matrix, adjacency list, in this project we have use adjacency list to implement the graph. Since graphs have developed applications in many fields different graphs algorithms have been developed. This project has implemented following algorithms:

- 1) Minimum Spanning Tree Algorithm
 - i) Prim's Algorithm
 - ii) Kruskal's Algorithm
 - iii) Boruvka's Algorithm
- 2) Single Source Shortest Path Algorithms
 - i) Dijkstra's Algorithm
 - ii) Bellman Ford's Algorithm
- 3) All-Pairs Shortest Path Algorithms
 - i) Floyd-Warshall Algorithm
- 4) Clustering Coefficient in Graph Theory
 - i) Average Cost of Local Clustering

Proposed System

The proposed provides a user-friendly user interface which allows user to select algorithms, input files and graph options. Firstly, user has two options either view graphs or run graph algorithms.

When view graph options is selected, user selects input file (no of nodes). The selected file is read, each node in the graph has x-y coordinates, there are edges in the graph and each edge has a bandwidth cost associated with it. When run algorithm option is selected user is provided a list of graph algorithms and a list of input files. After user selects a any graph algorithm and input file, input file is read and selected algorithm is executed. If selected algorithm is a minimum spanning tree (MST) algorithms user has option to view resultant MST graph, and if selected algorithm is shortest path algorithm user will have option to view resultant shortest paths from source to all other nodes.



Experimental Setup

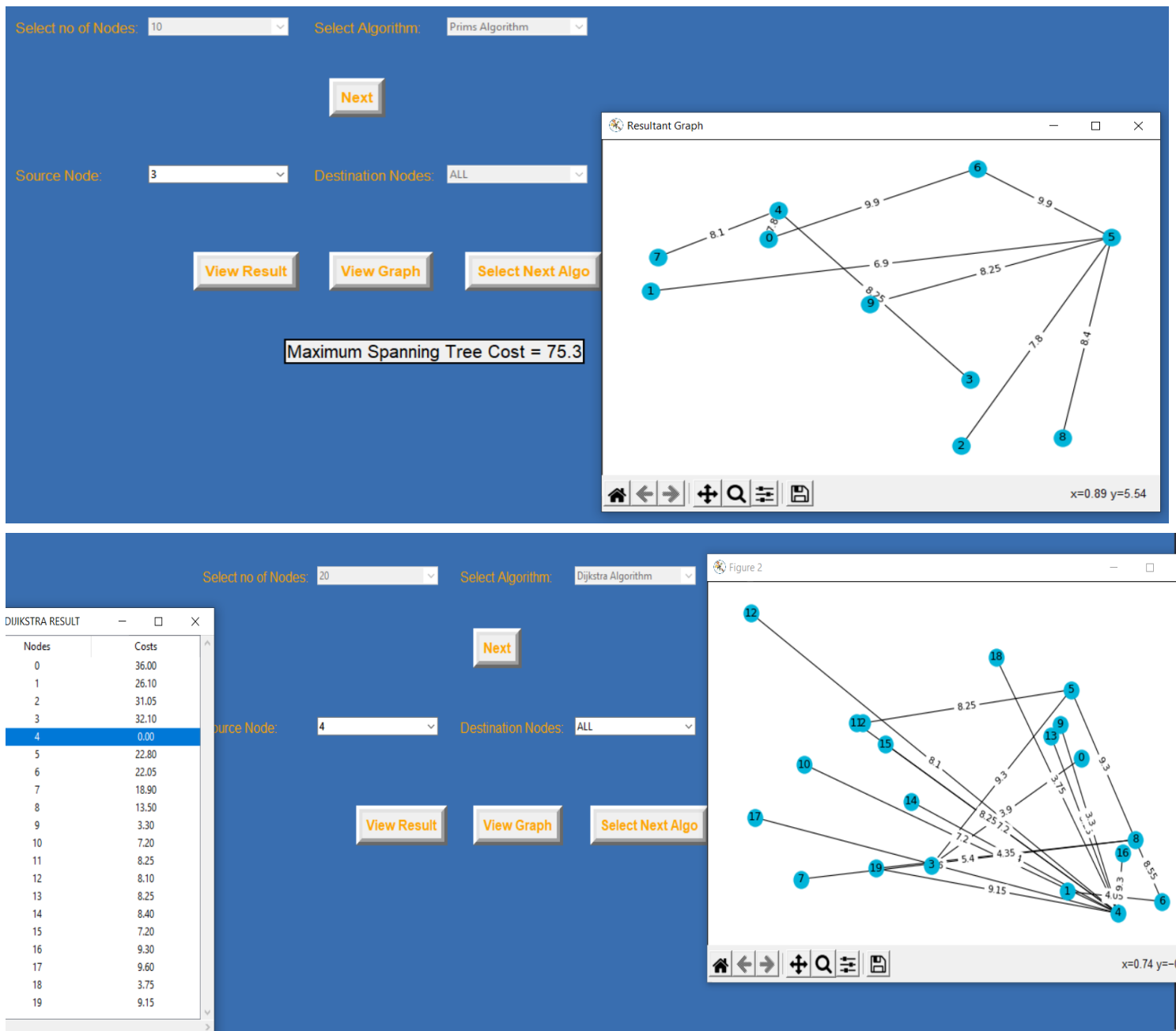
There are in total 10 input files with increasing no of nodes (10-100 nodes). Each input file contains count of total no of nodes, coordinates of each node, and edges between the nodes where each edge has an associated bandwidth cost. In total seven graph algorithms are run of all these input file and result are compared among minimum spanning tree and shortest path algorithms.

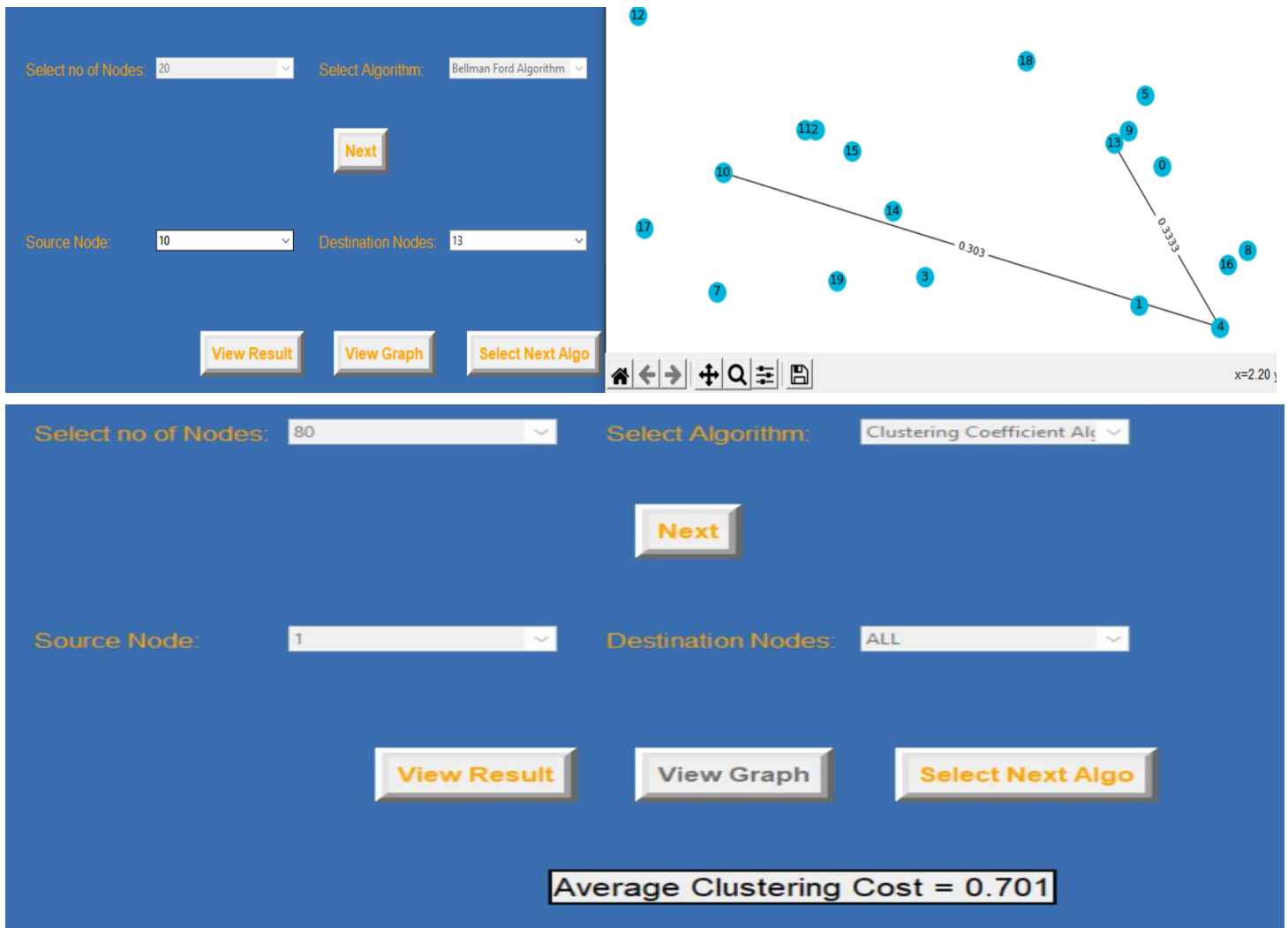
- Prims, Kruskal and Boruvka's are all minimum spanning tree algorithms they take Graph and a source node as input and produces a minimum cost spanning tree (tree with minimum no of edges).

- Dijkstra and Bellman Ford are single source shortest path algorithms, they take Graph and source node as input and provides the minimum cost path from source node to all other nodes in the graph. Dijkstra's algorithm follows greedy approach whereas Bellman Ford's algorithm follows dynamic programming approach.
- Floyd-Warshall is all-pair shortest path algorithm, it take Graph as input and provides minimum cost from all nodes to all other nodes in the graph. It follows dynamic programming approach.

Results And Discussion

The results are calculated and shown in tabular format for better readability. The resultant graphs are also created and displayed using networkx and matplotlib library for better visualization of results.





Conclusion

The project successfully demonstrates all above-mentioned graph algorithm on given benchmark files with increasing no of nodes. The results obtained are displayed in tabular format along with graphing utility for better understanding and visualization.

References

GitHub Repository

https://github.com/Noman-Vadsariya/Algo_project

NetworkX Documentation

<https://networkx.org/>

Tkinter Documentation for GUI

<https://tkdocs.com/>